School of Computing, Engineering, and Physical Sciences
Assessment Guidance Coversheet

Module Code:	COMP09041	
Module Title:	AI Programming for Games	
Module Co-ordinator:	Dr. Paul Keir	
Assessment set by:	Dr. Paul Keir	
Learning Outcomes Assessed:	L2 Investigate, via programming, the techniques covered in the course. L3 Select an appropriate technique for any desired aspect of intelligence L4 Apply these techniques within software relevant to videogame development	
Issued:	Tuesday, 24th March, 2026	
Deadline:	5pm, Friday, April 17th, 2026	
Feedback:	Friday, May 8th, 2026	
Individual / Group assessment:	Individual ❑	Group ☑

Anonymity

The University Regulations makes it clear that all assessments are marked anonymously (Regulation 3.4) unless the assessment itself renders anonymity impossible e.g. placements, presentations, practical assessments. Unless otherwise stated, do not put your name or Banner ID on your submission.

Referencing

The standard referencing style at UWS is **Cite Them Right (CTR) Harvard** and your references should be in accordance with these guidelines. The University provides a drop-down menu that lets you see examples of how you should reference journals, books, websites etc. Guidance on referencing styles can be found on the UWS Library website: https://uws-uk.libguides.com/referencing.

Extenuating Circumstances

The University recognises that, from time to time, you may encounter issues which may prevent you from being able to submit or undertake an assessment. Where this is the case, you can complete an Extenuating Circumstances Submission (ECS) for consideration. The ECS will be forwarded to the School Assessment Board to take account of this declaration in recording your module marks[1]. Guidance on ECS claims can be found: https://www.uws.ac.uk/current-students/supporting-your-studies/exams-assessment-appeals/academic-appeals-extenuating-circumstances.

[1]Note that any mark / grade you receive is provisional and may be subject to change until there has been internal moderation, external examination and ratification at the School Assessment Board.

Generative AI

The type of Generative AI you are allowed to use within your assessment is:

Type	Description	Allowed
1	**Restricted** Only the use of routine and established tools, such as auto-transcription, spell checkers, grammar check is permitted.	☑
2	**Specified** Generative AI may be used for clearly delineated tasks as appropriate / allowed / recommended, although its use is **not** mandatory in order to complete the assessment.	❑
3	**Open** No specific restrictions but with requirement to track key stages / tools utilised. The use of such tools is **not** mandatory in order to complete the assessment. This may include: (i) Socratic chatbot, (ii) summarisation, simplification, synthesis, and translation, (iii) generative illustrative media content, and (iv) support production of multimedia artefacts.	❑
4	**Embedded** Generative AI is a feature of the assessment itself. Here the use of Generative AI is a focal aspect of the assessment. This may include (i) using named tools for specific outcome, (ii) output comparison and critique, and (iii) develop or error check code.	❑

Assessment Student Declaration Coversheet

All students are expected to complete the Student Declaration and insert that as the first page of your submission. The Module Co-ordinator will have uploaded this to Aula for you to complete.

- For written assessments, insert the Declaration as the first page of your document.
- For assessments not in the written format (e.g. video, audio, presentation, or practical work), submit the Declaration as a separate file.
- For group assessments, unless otherwise directed by your lecturer, the group may submit a single shared declaration.

Second Assignment (Group Project)
AI Programming for Games
COMP09041

Issue Date:	Tuesday, 24th March, 2026
Due Date:	**5pm, Friday, April 17th, 2026**

Voice Interaction

In this assignment you will add voice interaction to a traditional 2D RPG game prototype. You are provided with an interactive SDL3 C++ program featuring the player (a Knight); the Grim Reaper; and the King. Three pickups (coloured sweets) are also provided.

This is a group project. Your team allocations are provided to you on Aula.

Your zipped submission (just one file) should include your source code and a video. Only one team member should submit.

Background

The provided `src/voice-interaction.cpp` file builds on the 2D SDL3 C++ game framework you have been using throughout previous labs. The CMakeLists.txt file includes configuration support for speech recognition (via **Whisper**); and speech generation (via the **ElevenLabs API**). The **aipfg** utility classes within the `src/aipfg` subfolder are also available, assisting with access to the LLM functionality of the UWS Polaris server.

You will though need to add the necessary C++ classes to the program in `voice-interaction.cpp`. For example, you should use the `SDL3_AudioRecorder` and `WhisperTranscriber` to allow the player to speak to the 2 non-player characters (NPCs); i.e. the Grim Reaper and the King. This is described in Week 8's lab. So too you should use the `ElevenLabs` class which was introduced in

Week 9's lab. Communication with the LLM provided by the Polaris server can use the handy `ChatClient` class introduced in Week 3's lab. As each of these operations will likely introduce a stall in the game loop, you should also use the `std::async` function template to mitigate the visibility of that to the player; as explained in Week 4's lab.

The Scenario

The game should pursue the following narrative:

To speak with an NPC, the player's avatar (the knight) should walk up to the NPC, and hold **Space**. While the space key is held, the player's voice should be captured using the microphone. When the space key is released, the NPC should reply using speech; the text of this is generated by the Polaris LLM, and made audible using the ElevenLabs API.

The King requires the Golden Fleece; the game is successfully completed when the King receives it. If the player speaks to the King, he will talk about his need for it. If the player has this item in their inventory, <u>and they offer it verbally</u>, they can give it to the King; e.g. *"Here, take the Golden Fleece"*. n.b. This should only succeed, if the player does indeed have it in their inventory.

The Grim Reaper (initially) has the Golden Fleece. If the player speaks to the Grim Reaper, he may talk about anything. If the player asks if he has the Golden Fleece, he will acknowledge that he has it. If the player requests it, the Grim Reaper will explain that he will only give away the Golden Fleece in exchange for 3 sweets. If the player has three sweets in their inventory, they can give them to the Grim Reaper, but (again) <u>only after offering them verbally</u>; e.g. *"Here, take the 3 sweets in exchange for the Golden Fleece"*. n.b. This should only succeed, if the player does indeed have the 3 sweets in their inventory.

The three sweets are arranged near to the player. When the player touches a sweet, it is picked up. ImGui should be used to display the number of sweets that the player has. ImGui should also be used to display the number of Goden Fleeces the player has (one at most).

To emphasise the use of the voice, only 5 keyboard controls should be used: move left; move right; move up; move down; and space to speak. The spoken response of an NPC can still be audible if the player walks away, but a new conversation can only start when the player's avatar is directly beside an NPC.

If you would rather the NPC start the conversation, you can add the use of another button to prompt them to speak.

Assignment Brief

Attempt the following 10 tasks.

1. Add an inventory HUD using ImGui to display the number of sweets and Golden Fleeces currently held by the player. The HUD should update in real time as items are collected or traded. **(4 points)**

2. Using the ElevenLabs website, create a distinct voice for each of the two NPCs (the Grim Reaper and the King). The voices should be appropriate to each character's personality and used consistently throughout the game. **(4 points)**

3. Integrate the Whisper API so that, when the player holds the Space key near an NPC, the microphone records their speech and Whisper transcribes it to text. The transcription should be accurate enough to support the game narrative. **(4 points)**

4. Send the transcribed speech text to the Polaris LLM server using the `ChatClient` class. Use an appropriate system prompt that gives each NPC its own personality and awareness of the game scenario. **(4 points)**

5. Receive and parse the response text returned by the Polaris LLM server. The response should be contextually appropriate to the NPC and the player's utterance, and should correctly handle the inventory logic (e.g. only allowing a trade when the player actually has the required items). **(4 points)**

6. Pass the NPC's response text to the ElevenLabs API so that it is spoken aloud using the NPC's voice. The spoken reply should play back without blocking the game loop. **(4 points)**

7. Implement the Grim Reaper trade: the Grim Reaper will only hand over the Golden Fleece after the player has verbally offered three sweets *and* actually has three sweets in their inventory. Verbal offers without the items should be declined by the Grim Reaper. **(4 points)**

8. Implement the King trade and game conclusion: the King will only accept the Golden Fleece after the player has verbally offered it *and* actually has it in their inventory. Successfully handing over the Golden Fleece should conclude the game. **(4 points)**

9. Ensure there is no noticeable freeze or pause in the game's visual activity (e.g. sprite animation or background rendering) while long operations — speech recording, Whisper transcription, LLM requests, and ElevenLabs synthesis — are in progress. This requires using `std::async` and `std::future`. **(4 points)**

10. Submit a video with voiceover (2-5 minutes) demonstrating your game in action: show the player collecting sweets, trading with the Grim Reaper,

and completing the game with the King. Upload the video to Aula alongside your zip submission. **(4 points)**

Resources

You are provided with the main C++ file `src/voice-interaction.cpp`, along with the utility files in the `src/aipfg` directory. A `CMakeLists.txt` is also provided. Use CMake and Visual Studio 2022 (or your preferred compiler) with VCPKG as in the labs.

Marking Scheme

The assignment is worth 40% of the marks awarded for the entire COMP09041 module. The following provides a summary breakdown of the marking scheme:

1. Add an inventory HUD using ImGui to count sweets/fleeces held	4
2. Create a voice for each of the NPCs using the ElevenLabs website	4
3. The Whisper API transcribes the player's speech to text	4
4. The Polaris LLM server receives the transcribed speech	4
5. The response text from the Polaris LLM server is received back	4
6. The NPC response text is spoken as an audible reply via the ElevenLabs API	4
7. The Grim Reaper provides the Golden Fleece *only* in exchange for 3 sweets	4
8. The King concludes the game *only* in exchange for the Golden Fleece	4
9. No noticeable freeze in the game while long operations are completed	4
10. A video with voiceover showing your game in action	4

Plagiarism

Ensure your work is developed only by your own team. You can discuss ideas with other teams, regarding how to prepare a solution, but the *copying or sharing of code is not permitted*.

Anonymity

Please use only the Banner IDs of your team members to identify yourselves in your submission. Ensure the Banner IDs of all team members appear in a comment at the top of your source code.